Digital Signal Processing Exercise Sheet 4

# Sampling

Script: Chapter 4 (Sampling and Quantisation)

**Learning goals**

- Derive and apply the quantisation-noise model and the rule SNR $\approx 6.02N + 1.76\,\mathrm{dB}$; size an ADC for the knock sensor.
- Configure timer-driven sampling on a microcontroller and verify the sampling rate and the conversion time with the logic analyzer.
- Quantify the effect of sampling-time jitter and compare it with quantisation noise.
- Resample a time-sampled engine signal onto a crank-angle grid using the edges of the 60-2 crank signal, and explain when the time domain and when the angle domain is the natural one.

## Setup: JSLinux

From this sheet on, the lab exercises also run in JSLinux, a Linux system in the browser. If you have not done the JSLinux bonus exercises of Sheets 0–2, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet04/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o quant quant.c -lm
./quant
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

## Background

An ADC with $N$ bits and full-scale range $V_{\mathrm{FS}}$ has the step size (1 LSB) $\Delta = V_{\mathrm{FS}}/2^N$. If the signal is "busy" (it moves over many steps between samples), the quantisation error $e = x_q - x$ is well modelled as white noise, uniformly distributed in $[-\Delta/2, \Delta/2)$. Sampling instants that deviate randomly from the ideal grid, $t_n = nT_s + \delta_n$ with $\mathrm{std}(\delta_n) = \sigma_j$ (*jitter*), produce an additional error $e_n \approx \dot{x}(nT_s)\,\delta_n$ that grows with the signal frequency.

An ECU samples two kinds of signals: *time-based* (fixed $T_s$, e.g. the knock sensor with 20 kHz to 50 kHz) and *angle-based* (one value per fixed crank-angle increment, triggered by the crank wheel). An angle grid can also be computed afterwards from time samples: the crank edges provide pairs (time, angle), between which the angle is interpolated.

### Exercise 4.1 Quantisation and jitter budget

PAPER

a) Assume a uniformly distributed quantisation error. Derive its variance and show that a full-scale sine quantised with $N$ bits has SNR $= 6.02N + 1.76\,\mathrm{dB}$. By how much does the SNR drop if the sine uses only 1/10 of the full-scale amplitude?

b) Compute 1 LSB and the rms quantisation noise for the Arduino Uno (10 bit, 0 V to 5 V) and the ESP32 (12 bit, 0 V to 3.3 V).

c) The conditioned knock-sensor signal spans 0 V to 5 V (offset 2.5 V) for the heaviest knock. The weakest knock that still has to be measured has an amplitude of 10 mV; its quantisation SNR must be at least 30 dB. Which dynamic range (in dB) is required, and how many bits does the ADC need?

d) The knock detector only uses a band of width $B = 1\,\text{kHz}$ around the knock frequency, the ADC samples at $f_s = 50\,\text{kHz}$. Assuming white quantisation noise, by how many dB does a band-pass filter improve the SNR of (c)? How many ADC bits does this "save"?

e) Show that jitter limits the SNR of a sine of frequency $f$ to $\text{SNR}_j = -20\log_{10}(2\pi f\sigma_j)$. (i) Which jitter is allowed for a 6.5 kHz knock signal if jitter shall not degrade a 12-bit ADC? (ii) A sampling loop that polls `micros()` (resolution 4 µs on the Uno) produces a timing error uniformly distributed over 4 µs. Compute $\sigma_j$, $\text{SNR}_j$ at 6.5 kHz and the *effective number of bits* $\text{ENOB} = (\text{SNR} - 1.76)/6.02$.

## Exercise 4.2 ADC in practice: timer-driven sampling

WOKWI

Open the Uno project `code/wokwi/sheet04_adc/` (sketch, diagram and the `engine-sim` chip files). Timer1 triggers an interrupt service routine (ISR) at $f_s = 5\,\text{kHz}$ that samples the `KNOCK` output of the chip (A1) into a RAM buffer of 400 samples; the buffer is printed afterwards (the UART cannot transmit 5 kHz × a CSV line in real time). Pin 8 toggles at every sample (logic analyzer D0), pin 9 is HIGH while the ISR runs (D1), the chip's `TDC` output (pin 2, INT0) marks combustion TDCs.

a) Timer1 runs in CTC mode with prescaler 8. Compute the compare value `OCR1A` for 5 kHz and complete `timer1_start()`. What is the lowest sampling rate possible with this prescaler?

b) Complete the ISR. Record about 0.1 s with the logic analyzer and determine from the VCD file (i) the sampling period (D0) and (ii) the time the ISR needs (pulse width on D1). Which sampling rate is the theoretical maximum for this structure? Why is the practical limit lower?

c) Between combustions, 140° to 175° after each TDC, the knock signal contains only sensor noise. Complete the computation of the index range of this quiet window (at 3000 rpm) and run the sketch with the noise slider at 50 mV, 10 mV and 0 mV. Compare the printed standard deviation (in LSB) with your expectation (chip output = 2.5 V + 0.6· sensor signal). Would the 12-bit ESP32 ADC improve the knock measurement here?

d) Turn the potentiometer to an arbitrary position and read the code several times. Is the quantisation error of this constant input "noise"? What does this mean for the validity of the model of Exercise 4.1?

> Wokwi measures time in *simulated* time; the Uno core is simulated instruction by instruction, but treat the measured durations as indicative.

## Exercise 4.3 Quantisation noise and jitter

JSLINUX

`code/jslinux/sheet04/quant.c` generates a sine of $f_0 = 1234.5\,\text{Hz}$ at $f_s = 50\,\text{kHz}$ ($10^5$ samples; $f_0/f_s$ is not a simple fraction, so the error is not periodic). The full-scale range is $[-1, 1)$.

a) Implement the function `quantise()`, a mid-tread quantiser with $N$ bits: rounding to the nearest step and clipping to the codes $q = -2^{N-1}, \ldots, 2^{N-1} - 1$. Implement `snr_db()`. Measure the SNR for $N = 4 \ldots 16$ at full scale and compare with $6.02N + 1.76\,\text{dB}$. For which $N$ does the formula fail, and why?

b) Repeat with an amplitude of 0.1 (−20 dBFS).

c) Implement jittered sampling (Gaussian $\delta_n$, no quantisation) and measure the SNR for $f = 100\,\text{Hz} \ldots 20\,\text{kHz}$ and $\sigma_j = 0.1, 1, 4\,\mu\text{s}$. Compare with $-20\log_{10}(2\pi f \sigma_j)$.

d) Combine a 12-bit quantiser with jitter: compute the ENOB for the knock frequency 6.5 kHz. At which signal frequency are jitter noise and quantisation noise equal for $\sigma_j = 1\,\mu\text{s}$?

## Exercise 4.4 Time domain vs. crank-angle domain JSLINUX

In `code/jslinux/sheet04/angle_resample.c` the virtual engine accelerates from 1500 rpm to 4500 rpm within 1 s (`e.rpm` is changed during the simulation). The ion-current signal is sampled at $f_s = 50\,\text{kHz}$; the rising crank edges are time-stamped with a resolution of 2 µs ("input capture" as in a real ECU), and the cam level is latched at every edge.

a) Assign a crank angle to every edge: detect the gap (edge interval $> 2\times$ the previous one), give the first edge after the first gap the angle 0° or 360° (cam level!), then add 6° per edge, 18° across the gap.

b) Resample the signal onto the grid $\theta_m = \theta_0 + m \cdot 0.5°$: for every $\theta_m$ interpolate the time $t(\theta_m)$ linearly between the two neighbouring edges, then the signal value $x(t(\theta_m))$ linearly between two time samples. The program compares the grid with the true model angle: report the maximum and rms angle error and explain its size. Rebuild with `-DDT_CAP=20e-6` (edges only known at the sample instants) and compare.

c) The program correlates every engine cycle with the first one, (i) in the angle domain (1440 grid points per cycle) and (ii) in the time domain (segments with the sample count of the first cycle, each starting exactly at its cycle start). Interpret the table and the two plots.

d) The knock-sensor signal contains crank-synchronous parts (combustion vibration $0 \ldots 60°$ after TDC) and fixed-frequency parts (valve impacts: 8 kHz, $\tau = 0.4\,\text{ms}$). How do the two look in the angle domain during the ramp? Compute the angular length of $3\tau$ of a valve burst at 1500 rpm and 4500 rpm.

e) Which angle step $\Delta\theta$ would be needed to sample the 10.5 kHz knock mode directly in the angle domain at 600 rpm and at 6000 rpm? Which signals should an ECU therefore sample angle-based, which time-based?

**Deliverables**

4.1: derivations and numbers. 4.2: completed sketch, logic-analyzer screenshot with sampling period and ISR duration, quiet-window results for three noise levels. 4.3: SNR tables (quantisation, jitter, ENOB) with short discussion. 4.4: completed program, angle-error values for two capture resolutions, the correlation table and the two plots with your interpretation.